

Python 觉醒



予你以梦

版权与许可声明

Copyright © [2026] [绘萤者 Lynvortex]

代码部分：MIT 许可证

本书示例代码采用 MIT 许可证授权。

MIT License

Copyright (c) [2026] [绘萤者 Lynvortex]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND.

文字与图片：CC BY-NC-ND 4.0

本书文字、插图和图片采用知识共享 署名-非商业使用-禁止演绎 4.0 国际许可协议授权。

您可以：自由复制、分发本作品。

须遵守：

署名（注明作者及本许可）

非商业使用

禁止修改或制作衍生作品

> 完整许可证文本：<https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode.zh-Hans>

最终解释：本声明页中对许可证的说明仅为便于理解，不构成法律意见。各许可证的完整条款以原文为准。若有任何歧义，以 MIT 许可证原文及 CC BY-NC-ND 4.0 官方法律文本为准。

造梦者的第一行诗

嗨，别来无恙啊！

编程并不神秘，它更像是一门与电脑对话的手艺。而 Python，是这门手艺中最亲切、最顺手的工具之一——语法清晰、功能强大，不论你是完全零基础的初学者，还是想转行到数据科学或 Web 开发的朋友，都能比较轻松地迈过门槛。

每章末尾的「自检」是给你爬完一座山之后回头看一眼——确保自己没掉队。每题只有一行提示方向的关键词，没有答案。因为你需要的不是“我的答案对不对”，而是“跑一下就知道对不对”。

后面附了一份「动手才是真的学」。那里没有“题目”，只有“试试看”和“搞坏它”。你不需要全部做完。编程里最骗人的一句话是“我看懂了”。代码只有亲手敲过、敲错过、再改对，才是真正属于你的。

为了让这本书能被更多人自由地使用和分享，我做了一个不寻常的决定：

书中的示例代码全部采用 MIT 许可证，你可以毫无顾虑地复制、修改，甚至用到自己的商业项目中。

书中的文字和图片 则采用 CC BY-NC-ND 4.0 许可证，你可以免费分享这本书（电子版），但请保留我的署名，不要用于商业目的，也不要修改书中的内容。

Github 仓库：<https://github.com/lynvortex/Python-rise>

这样的选择，是希望知识本身尽量开放，同时尊重作者的心血。如果你在某个论坛、网盘或读书群里见到了这本书的完整副本，我的目的就达到了。

最后，也是最重要的：编程是一项需要动手的技能。请不要只“看”这本书，一定要把每一个示例亲手敲进电脑里，甚至故意改坏它，看看会发生什么——出错、调试、再运行，这才是真正的学习。

祝你编码愉快，也期待你早日写出自己的第一个程序。

绘萤者

2026.6.8

目录

第 1 章 开始之前：搭建你的思维实验环境.....	- 1 -
1.1 安装 Python：就像装个游戏.....	- 1 -
1.2 pip：Python 的"应用商店".....	- 2 -
1.2.1 常见问题：pip 找不到.....	- 2 -
1.2.2 提速 pip：配置国内镜像源.....	- 3 -
1.3 第一次亲密接触：让电脑帮你算数.....	- 4 -
1.4 解释器、编辑器、IDE：三个容易搞混的概念.....	- 4 -
1.5 可选择的工作台：VSCode 极简上手.....	- 5 -
1.6 动手：创建并运行你的第一个.py 脚本.....	- 5 -
1.7 总结与下一步.....	- 7 -
自检.....	- 7 -
第 2 章 万物皆对象：先学会用积木.....	- 9 -
2.1 数字：最诚实的积木.....	- 9 -
2.2 字符串：会说话的文字积木.....	- 10 -
2.3 布尔值：真假美猴王.....	- 11 -
2.4 None：空无一物也是一种积木.....	- 12 -
2.5 变量：贴上去的标签.....	- 12 -
1.Python 先在内存里创建了一个整数对象 42。.....	- 12 -
2.然后，它把一个叫 x 的标签贴在了 42 上面。.....	- 12 -
3.以后你用 x，Python 就顺着标签找到 42。.....	- 12 -
变量命名规则.....	- 13 -
2.6 type() 函数：照妖镜.....	- 13 -
2.7 总结与下一步.....	- 13 -
自检.....	- 14 -
第 3 章 会思考的代码：让程序做判断.....	- 15 -
3.1 一个真实的问题：我能玩游戏吗？.....	- 15 -
3.2 if 语句的结构拆解.....	- 15 -
第一部分：关键字 if.....	- 15 -
第二部分：条件.....	- 15 -
第三部分：缩进的代码块.....	- 16 -
第四部分：else（可选）.....	- 16 -
3.3 多岔路口：if-elif-else.....	- 16 -
3.4 条件嵌套：岔路里面还有岔路.....	- 18 -
3.5 条件表达式（进阶选读）.....	- 18 -
3.6 真值测试：什么条件算是“真”？.....	- 19 -

3.7 实战：写一个体重指数计算器（BMI）	- 20 -
3.8 总结与下一步	- 20 -
练习	- 21 -
第 4 章 重复的事交给机器：循环的乐趣	- 22 -
4.1 一个真实的问题：怎么做 100 个俯卧撑？	- 22 -
4.2 for 循环的结构拆解	- 22 -
4.3 for 不只是遍历数字	- 23 -
4.4 while 循环：条件不满足就一直跑	- 23 -
4.5 死循环：一个必须认识的幽灵	- 24 -
4.6 break 和 continue：循环里的特权操作	- 25 -
4.7 循环嵌套：循环里面套循环	- 25 -
4.8 实战：统计班级成绩	- 25 -
4.9 for 和 while 怎么选？	- 26 -
4.10 总结与下一步	- 27 -
自检	- 27 -
第 5 章 组装你的第一套工具：函数	- 28 -
5.1 一个真实的问题：为什么复制粘贴三次就该疯了？	- 28 -
5.2 定义和调用：给一段代码取名字	- 28 -
5.3 参数：让函数能接受输入	- 29 -
5.4 返回值：让函数能输出结果	- 30 -
5.5 无返回值的函数，其实是返回 None	- 31 -
5.6 文档字符串：写给未来自己看的信	- 31 -
5.7 参数的几种形态	- 32 -
5.8 作用域初探：函数内外是两套世界	- 33 -
5.9 实战：把 BMI 计算器封装成工具函数	- 34 -
5.10 总结与下一步	- 35 -
自检	- 36 -
第 6 章 数据的家：列表、元组、字典、集合深入	- 36 -
6.1 列表：最自由的序列	- 37 -
6.1.1 索引和切片：精准定位	- 37 -
6.1.2 修改、增加、删除	- 37 -
6.1.3 列表推导式：一行生成列表	- 38 -
6.2 元组：不可变的序列	- 38 -
6.3 字典：现实世界的映射表	- 39 -
6.4 集合：不重复的无序容器	- 40 -
6.5 四种容器对比与选择	- 41 -

6.6 深入理解：标签模型与可变对象	- 42 -
6.7 实战：写一个简单通讯录	- 42 -
6.8 总结与下一步	- 43 -
自检	- 43 -
第 7 章 文件与异常：跟外部世界对话	- 44 -
7.1 为什么需要文件和异常？	- 44 -
7.2 打开文件：建立跟外部世界的通道	- 44 -
7.3 with 语句：自动关门的管家	- 45 -
7.4 读文件：把文件内容变成字符串或列表	- 45 -
7.5 写文件：把程序的成果存下来	- 46 -
7.6 异常处理：给程序上保险	- 46 -
7.7 实战：统计小说词频	- 48 -
7.8 常见异常清单	- 49 -
7.9 总结与下一步	- 50 -
自检	- 50 -
附录 A GitHub 共享指南：把你的代码放到全世界的舞台上	- 51 -
A.1 Git 和 GitHub：一个是工具，一个是平台	- 51 -
A.2 安装 Git	- 51 -
A.3 告诉 Git 你是谁	- 51 -
A.4 创建 GitHub 仓库	- 52 -
A.4.1 无法正常访问 Github	- 52 -
A.5 把本地项目变成 Git 仓库	- 52 -
A.6 把本地仓库推到 GitHub (push)	- 53 -
A.7 日常修改流程：改代码 → add → commit → push	- 54 -
A.8 从 GitHub 拉取更新 (pull)	- 54 -
A.9 常用的周边操作	- 55 -
A.10 分支简介：同时开发多个功能不打架	- 55 -
A.11 总结	- 56 -
动手才是真的学	- 57 -
第 2 章实验：认积木	- 57 -
第 4 章实验：让机器重复	- 59 -
第 5 章实验：造工具	- 59 -
第 6 章实验：容器玩法	- 60 -
第 7 章实验：跟文件对话	- 61 -
综合挑战（学完第 7 章后做）	- 62 -

第 1 章 开始之前：搭建你的思维实验环境

搭建环境这件事，本不该成为你的第一道坎。把它想象成——你买了一台游戏机，总得先接上电源、插上手柄吧？这一章，我们就来做这件事。目标只有一个：让你亲手运行第一行 Python 代码，并且明白背后发生了什么。

放心，我不会扔给你一堆链接让你自己去踩坑。跟着我，一步一步来。

1.1 安装 Python：就像装个游戏

Python 的官方网站是 python.org。有个常见的坑：不是最新版就最好，但也不必选择太老的版本。当前主流稳定版本是 Python 3.12.x 或 3.13.x。绝大多数第三方库已经兼容这些版本，同时它们包含了最新的语法特性和性能优化。如果你遇到个别库不兼容的情况（概率极低），可以通过虚拟环境（将在后续章节学习）或临时降级解决，但不必一开始就选择旧版。

本书推荐：Python 3.12.x。它足够新，又有着广泛的库兼容性。

小贴士：版本号怎么看？

> Python 版本号形如 '3.12.4'：

> '3' 是大版本（Python 2 已停止维护，认准 3）

> '12' 是次版本（新特性在此引入）

> '4' 是修订号（只有 bug 修复）

> 本书代码在 3.12 和任何更高版本都能运行。当你学完基础，再升级版本只需重新安装 Python 即可，所有知识仍然适用。

下载好安装包后，双击运行。**最关键的一步**：窗口底部有一个复选框 'Add Python to PATH'，一定要勾上。这就像把 Python 加进系统的 '通讯录'，以后你可以在任何角落呼叫它。

点击 'Install Now'，等待进度条走完。好了。

安装完了，怎么知道成没成功？我们来验证一下。

打开命令终端：

Windows:按下 'Win + R'，输入 'cmd'，回车。在跳出来的黑色或白色窗口里，输入：

```
python --version
```


macOS/Linux: 打开终端 (macOS: Cmd+空格 搜索“终端”; Linux: Ctrl+Alt+T)。使用 `python3 --version` 检查 Python, 后续命令用 `python3` 和 `pip3` 替代文档中的 `python/pip`。

如果你看到类似 'Python 3.12.4' 的字样, 恭喜, 安装成功。这意味着你的操作系统已经认识 'python' 这个指令了。

1.2 pip: Python 的"应用商店"

`pip` 是 Python 的包管理工具, 用于查找、下载、安装和卸载 Python 包。它是 Python 开发者必备的工具之一, 特别是当你需要安装和管理不属于 Python 标准库的其他软件包时。

你现在也许用不上, 但还是有必要记一记。

常用命令:

安装包: `pip install 包名`

安装指定版本: `pip install 包名==1.0.0`

升级包: `pip install --upgrade 包名`

卸载包: `pip uninstall 包名`

查看已安装包: `pip list`

查看包详情: `pip show 包名`

1.2.1 常见问题: pip 找不到

这个通常有两个原因:

1. 安装时没勾 'Add Python to PATH'——这是排第一的元凶。PATH 是系统的“寻路地图”, 没勾那个框, 系统就不知道 Python 和 pip 装在哪。

2. 装了, 但命令行窗口没重启——如果你刚装完或刚改过环境变量, 之前打开的终端窗口不会自动刷新, 需要关闭重开。

方案一: 用 `'python -m pip'`

这是最稳妥的调用方式, 永远不会出现'找不到'的问题。无论 PATH 有没有配好, 只要 `'python'` 命令能跑, 这个就能跑:

```
python -m pip install requests
```

'-m'的意思是“把后面的模块当作脚本来运行”。翻译成成人话: Python 大哥, 你自己

去找一下 pip 放在哪,然后执行它。你不需要知道路径,系统也不需要配 PATH,全是 Python 内部的事情。

方案二：检查 PATH 环境变量

如果你想彻底修好 pip 命令,让它以后能直接用,得确认 PATH 是否包含了 Python 的安装路径和 Scripts 文件夹。

先找到 Python 装在哪:在命令行输 where python,会返回一个路径,例如:

```
C:\Users\你的用户名\AppData\Local\Programs\Python\Python312\python.exe
```

复制它。

检查系统 PATH:

按 Win 键,搜“环境变量”,点击“编辑系统环境变量”→“环境变量”→在“系统变量”里找到 Path,双击,看列表里有没有 ...Python312\ 和 ...Python312\Scripts\ 这两条。如果没有,新建添加。

改完后一定要关掉所有命令行窗口,重新打开一个,变动才会生效。

方案三：重新安装 Python,这次勾上 PATH

如果第二步不能解决问题,最简单暴力的方式就是直接重装 Python。这次在安装窗口底部,一定勾选'Add Python to PATH',然后点 Install。装完之后,关掉旧终端、开新终端,一切就正常了。

1.2.2 提速 pip: 配置国内镜像源

pip 默认从国外的 PyPI 服务器下载包,在国内直接使用可能只有几 KB/s,甚至超时断开。一个不需要任何技术含量的办法就是改用国内的镜像站,它们是 PyPI 的完整复制,下载速度可以跑满你的宽带。

常用镜像地址(任选其一):

清华大学: 'https://pypi.tuna.tsinghua.edu.cn/simple'

阿里云: 'https://mirrors.aliyun.com/pypi/simple'

中科大: 'https://pypi.mirrors.ustc.edu.cn/simple'

从 pip 10.0 开始,支持用 pip config set 命令直接配置,无需手动编辑文件。打开命令行,执行:

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

再执行:

```
pip config set install.trusted-host pypi.tuna.tsinghua.edu.cn
```

验证是否生效：

```
pip config list
```

如果输出中包含你刚设置的 `global.index-url`，就说明永久配置成功了。以后再执行任何 `pip install` 包名，都会自动到镜像站下载，速度起飞。

1.3 第一次亲密接触：让电脑帮你算数

现在，别走开，我们立刻在里面写点东西。在同一个命令行窗口输入 `'python'`，然后按回车。你会发现光标前面多了 `'>>>'` 这个符号。

这就进入了 Python 的交互模式。简单说，就是你敲一行命令，它立刻给你回答，像在跟一个活的翻译官对话。

我们试试让它做道算术题。在 `'>>>'` 后面输入：

```
print(2 + 3)
```

然后按回车。屏幕上立刻出现了：5

看，你刚刚给电脑下达了一个指令：“计算 2 加 3，并把结果打印出来。” 它完美执行了。

别小看这个 `'5'`。这是你作为程序员的第一行有效输出。很多人的编程之路，就是从这样一个简单的加法开始的。你再试试：

```
print("Hello, world!")
```

屏幕上会出现 `'Hello, world!'`。注意到了吗？计算数字时，直接写 `'2+3'`；而要让电脑处理文字，得用引号把文字包起来。这是为了让电脑区分“命令”和“内容”，我们下一章会细讲。

现在，你可以尽情玩一会儿。把 `'2+3'` 换成别的数字，或者把引号里的字改掉。如果想退出这个交互模式，输入 `'exit()'` 或者直接关掉窗口就行。

> 留一个印象：两种运行代码的方式

> 你刚刚体验的是交互模式，输一行，执行一行。还有一种更常见的方式，是把代码写在一个文件里，一次性执行。这就是脚本模式。两种模式你会反复用到，我们在 1.7 节会动手试一试。

1.4 解释器、编辑器、IDE：三个容易搞混的概念

刚才你已经在交互模式里写了代码。现在你可能想问：那些专业程序员是在这种黑窗

口里写程序吗？当然不是。这里就要厘清几个概念，它们常常让新手迷惑。

解释器：就是刚才响应你'python' 命令的那个程序。它负责把你写的 Python 代码，翻译成电脑能执行的机器指令。你写的文本，它来“解释”。

编辑器：任何能编辑纯文本的工具，比如 Windows 自带的记事本，Mac 的文本编辑。你可以把代码写在一个'.py'后缀的文件里，然后用'python 文件名.py'命令去让解释器执行整个文件。但纯文本编辑器太简陋了，连代码高亮都没有。

IDE（集成开发环境）：就是加强版的编辑器。它把“写代码”、“运行”、“调试”等功能都集成在一起，像给程序员准备的豪华工作台。PyCharm、VS Code 就是著名的 IDE。

你可能会问：那我该用哪个？新手最大的痛苦来源于工具复杂度远超学习内容本身。

1.5 可选择的工作台：VSCode 极简上手

VSCode (Visual Studio Code)是目前最流行的免费代码编辑器，对 Python 的支持极佳。最妙的是，它既能编辑'.py'脚本，也能直接运行。所以，你完全可以只装一个 VSCode。

很多专业程序员每天都在 VSCode 里工作，尽早熟悉它，你就走在了一条最踏实的路上。

1.去 <https://code.visualstudio.com> 下载安装 VSCode。它就像装一个普通软件，一路默认即可。

2.打开 VSCode，点击左侧“扩展”图标（或按 'Ctrl+Shift+X'），搜索'Python'，安装微软官方的 Python 扩展。这个扩展会为你提供代码补全、错误提示等智能功能。

3.新建一个文件，保存为'hello.py'，输入：

```
print("Hello World")
```

按下'Ctrl+s'进行保存，点击右上角的“运行”按钮（▶），或按'Ctrl+F5'，下方终端会显示出结果。

VSCode 的终端就类似咱们刚刚用的命令行，在这里你也可以直接输入'python'进入交互模式。这意味着，VSCode 把编辑器、终端、文件管理都整合在了一起。如果你是那种喜欢“一个窗口搞定一切”的学习者，用 VSCode 作为起点也完全可行。

1.6 动手：创建并运行你的第一个.py 脚本

光在命令行里输一行跑一行，成不了真正的程序。真正的程序是一系列指令，保存在一个'.py'文件里，随时可以重复执行。'.py'文件本质上就是一个纯文本文件，只是后缀名告诉系统：“这是 Python 代码。”

接下来，我们走一遍最简单、最不容易出错的流程：使用 VSCode 新建文件。

第一步：在 VSCode 中新建文件

打开 VSCode，按 `Ctrl + N`（Mac 上用 `Cmd + N`）新建一个空白文件，按 `Ctrl + S` 保存，弹出一个保存对话框，选择一个你容易找到的文件夹，比如桌面的 `python_learning`（没有的话可以新建一个）。

文件名输入 `welcome.py` —— 重点：结尾必须是 `.py`，VSCode 会自动识别这是 Python 代码。

点击“保存”。

小提示：你不需要手动在文件夹里“新建文本文档”再改后缀。直接在 VSCode 里保存为 `.py` 最干净，也不会踩“文件名变成 `welcome.py.txt`”的坑。

第二步：告诉 VSCode 使用哪个 Python（只需做一次）

在你写代码之前，最好先确认 VSCode 已经“认识”你刚安装的 Python。这一步通常只需要做一次。

按 `Ctrl+Shift+P`（Mac 用 `Cmd+Shift+P`），打开命令面板。

输入 `Python: Select Interpreter` 并选择。

在弹出的列表里，选中你安装的 Python 版本，比如 `Python 3.12.x`。

如果列表里空空如也，说明 Python 没装好或 PATH 没配好——回到 1.1 节检查一下。

第三步：写代码

在 VSCode 的编辑区输入以下代码：

```
name = input("请输入你的名字：")
greeting = "你好，" + name + "! 欢迎来到 Python 的世界。"
print(greeting)
```

4.按'`Ctrl + S`'保存。

看到了吗？VSCode 会自动识别 `.py` 后缀，为你提供语法高亮和智能提示。你只需要一个能写代码的编辑器，记事本不需要参与“写代码”这个环节。

第四步：运行它

VSCode 里有好几种运行方式，挑你顺手的：

方法一（最简单）：点击编辑区右上角的 ▶ 运行按钮。

方法二（快捷键）：按 `Ctrl + F5`（Mac 用 `Ctrl + Fn + F5` 或 `Cmd + F5`，取决于设置）。

方法三（使用终端）：按 `Ctrl + ``（反引号，键盘 `Esc` 下面的那个键）打开内置终端，然后输入：

```
python welcome.py
```

注意：在 macOS/Linux 上，可能需要用 `python3 welcome.py` 而不是 `python`。如果 `python` 提示找不到，就试试 `python3`。

无论哪种方式，程序都会提示你输入名字。键入名字后按回车，它就会热情地跟你打招呼。

这里发生了什么？

- `'input()'` 让程序暂停，等你输入。
- 你输入的文字被放进变量 `'name'`。
- `'+'` 把几段文字拼接成完整的问候语，存入 `'greeting'`。
- `'print()'` 输出最后结果。

虽然这些语法还没讲，但你已经模糊地感知到了：程序就是从上到下一步步执行的，你可以让程序等你，然后根据你的输入做出不同的反应。

1.7 总结与下一步

在这一章，我们没有急着学语法，而是做了一件重要的事：铺设跑道。你安装了 Python 3.12，理解了工具的区别，搭建了 VSCode 工作环境，学会了 `.py` 脚本的创建与运行方法，并建立了运行原理的初步概念。

从下一章开始，每学一个新知识点，你可以写进 `.py` 文件里跑一遍。这个练习的习惯，会让你对概念的理解深得多。

现在，你的电脑里已经有了完备的编程实验环境。下一章，我们将走进 Python 的对象世界。你会发现，你操作的每一个数字、每一段文字，都是一个活的“积木”，它们自带功能，你只需学会如何组合它们。

你跑起来了。欢迎来到编程世界。

自检

1. 试试 `'print(10 - 7)'` 和 `'print(10 * 7)'`。`*`是乘法，除法符号是什么？自己搜索一下看看。
2. 试着 `'print("你的名字")'`，把名字换成你自己的。（提示：引号内替换）

3.将练习 1 中的算术运算写成'calc.py'脚本，分别打印加减乘除的结果，然后在命令行运行它。注意体会“写完——保存——运行——看结果”的完整循环。

（提示：需要先“cd 文件所在目录”然后输入 `python calc.py` 运行）

第 2 章 万物皆对象：先学会用积木

在上一章，我们搭建好了工作环境，也跑了人生第一行代码。现在，你面对 VSCode 的编辑区，可能会想：我能写什么？我能操作什么？

答案很简单：一切皆是对象，一切皆是积木。

Python 世界里，你接触到的每一个东西——一个数字、一段文字、一个列表——都是一个“对象”。这个称呼听起来有点玄，但你可以这样理解：每个对象都是一块自带说明书的乐高积木，它知道自己是什么类型，能做哪些操作。你作为程序员，就是学习识别这些积木的形状，然后把它们拼装起来。

这一章，我们从最基础的积木开始：数字、文字、真假值，以及用来贴标签的变量。

2.1 数字：最诚实的积木

打开你的 VSCode 终端，进入交互模式。输入一个数字，按回车：

```
42
```

Python 什么也没抱怨，直接回你一个 42。它认识数字。你再试试：

```
3.14
```

```
-7
```

```
0
```

这些都是数字。Python 把数字主要分成三类，对应我们生活中的数学概念：

整数（int）：42、0、-7、1000000。Python 3 的整数可以无限大，不用担心溢出。

浮点数（float）：带小数点的，3.14、-0.5、1.0。即使写成 1.0，它也是浮点数，不是整数。

复数（complex）：1+2j，工科生可能用到，我们这本书不深入。

数字对象自带运算能力。加 +，减 -，乘 *，除 /，你已经见过了：

```
print(10 + 3) # 13
```

```
print(10 - 3) # 7
```

```
print(10 * 3) # 30
```

```
print(10 / 3) # 3.3333333333333335
```

新手坑位：除法的结果总是浮点数。

即使 $10/2$ ，结果也是 5.0 而不是 5。如果你想要整数结果（比如分东西，只要完整的份数），Python 提供了两个工具：

```
print(10 // 3) # 3, 整除（地板除），向下取整丢掉小数
print(10 % 3) # 1, 取余数（模运算），10 除以 3 剩 1
```

另外还有一个 `**`，代表乘方：

```
print(2 ** 10) # 1024, 2 的 10 次方
```

运算优先级和我们小学数学一样：先乘方，再乘除，再加减，括号里的优先。不确定的时候，加括号就对了：

```
print((2 + 3) * 4) # 20, 而不是 14
```

2.2 字符串：会说话的文字积木

数字是冰冷的，文字才有温度。在 Python 里表示文字，得用引号包起来。这样 Python 才能区分：这是要处理的“内容”，而不是“命令”。

```
"Hello"
'世界'
```

单引号和双引号都可以，但必须成对出现。它们造出来的，就是一个字符串（str）对象。

新手坑位：中英文引号是两回事。

编程里的引号必须是英文输入法下的“`"`或`'`”。中文的“和”Python 不认识，会直接报 `SyntaxError`。这是初学者最容易犯的错误之一，而且很多时候肉眼看不出来。如果代码莫名报错，先检查引号是不是英文的。

字符串可以和数字一样，用 `print()` 输出：

```
print("Hello, world!")
```

字符串的拼接

两个字符串可以用 `+` 粘在一起：

```
print("你好" + "世界") # 你好世界
```

还可以用 `*` 重复：

```
print("哈" * 5) # 哈哈哈哈哈
```

新手坑位：字符串和数字不能直接拼接。

如果你写 "我今年" + 25 + "岁"，会报错。因为 25 是整数，不是字符串。你需要先把它转成字符串——用 `str()` 函数：

```
print("我今年" + str(25) + "岁") # 正确
```

`str()` 能把很多东西变成字符串，它是一个类型转换函数，后面我们会经常用到。

`f-string`：最舒服的拼接方式

从 Python 3.6 开始，有一种更方便的写法，叫 `f-string`（格式化字符串）。在引号前加个 `f`，然后用花括号 `{}` 把变量或表达式包起来，就能直接嵌入：

```
age = 25

print(f"我今年{age}岁") # 我今年 25 岁

print(f"明年我就{age + 1}岁了") # 明年我就 26 岁了
```

`f-string` 会在后续章节大量使用，它比 `+` 拼接更直观、更不容易出错。现在你只需要记住：遇到“把东西塞进一句话里”的需求，用 `f-string`。

2.3 布尔值：真假美猴王

有些时候，我们只需要回答“是”或“否”。比如：登录成功了吗？分数及格了吗？这时候就用到了布尔值（`bool`）。

它只有两个成员：

```
True

False
```

注意首字母要大写，`true` 或 `TRUE` 都不对。Python 的大小写是严格区分的。

布尔值本身看着简单，但它们通常来自比较运算：

```
print(10 > 5) # True

print(10 < 5) # False

print(10 == 10) # True, 两个等号表示“比较是否相等”

print(10 != 10) # False, != 表示“不等于”
```

新手坑位：一个等号 `=` 和两个等号 `==` 是完全不同的东西。

`=` 是赋值（把右边的东西赋给左边），`==` 是比较（左边和右边是否相等）。这个坑几乎每个人都踩过，如果发现程序行为诡异，先检查是不是把 `==` 写成了 `=`。

`and`、`or`、`not` 可以把布尔值组合起来：

```
print(10 > 5 and 10 < 20) # True, 两边都真才真
print(10 > 5 or 10 > 20) # True, 一边真就真
print(not 10 > 5) # False, 取反
```

布尔值在下一章做判断时会大放异彩。现在你只需要知道：它们是“判断结果”的自然表达。

2.4 None：空无一物也是一种积木

有时候，你需要表达“什么都没有”或“暂未定义”。Python 里用 None 表示这个状态，它是一个特殊的对象，类型是 NoneType。

```
result = None
print(result) # None
```

None 不是 0，不是空字符串，也不是 False。它就是一种独特的“空瓶”状态。以后处理函数返回值、占位初始化时，None 会经常出现。初学时你先认识它，知道它叫“空值”即可。

2.5 变量：贴上去的标签

到现在，我们玩的数字、字符串都是“用完即丢”。但如果我想记住一个中间结果，后面再用，怎么办？这就需要变量。

变量的本质：标签，不是盒子

可以先把变量想象成一个“标签”贴在对象上，而不是一个装东西的“盒子”。这个比喻的区别，等你学到第 6 章的列表时就会亲眼看到。

Python 的变量更像一个标签，贴在对象上。

```
x = 42
```

这行代码应该这样理解：

1. Python 先在内存里创建了一个整数对象 42。
2. 然后，它把一个叫 x 的标签贴在了 42 上面。
3. 以后你用 x，Python 就顺着标签找到 42。

再来看：

```
y = x
```

这不是"复制一份",而是在 42 这个对象上再贴一个标签 $y \circ x$ 和 y 都指向同一个 42。

对于数字和字符串,两种理解暂时看不出差别。第 6 章讲列表时,你就能亲眼看到区别。先记住"标签"这个直觉就好。

变量命名规则

给变量起名字有几点限制:

- 1.只能由字母、数字、下划线组成,不能有空格。
- 2.不能以数字开头 (1name 不行, name1 可以)。
- 3.不能是 Python 的保留字,比如 if、for、class。这些词 Python 自己用了。

惯例: Python 社区推荐用 snake_case (蛇形命名法),全部小写,用下划线分隔:

```
user_name = "张三"
total_score = 95
```

驼峰命名 userName 虽然也能跑,但在 Python 里不推荐。保持一致,以后合作会更顺畅。

2.6 type() 函数:照妖镜

你拿到一个对象,不确定它是什么类型?别猜,直接问 Python。type() 函数就是那把“照妖镜”:

```
print(type(42)) # <class 'int'>
print(type(3.14)) # <class 'float'>
print(type("hello")) # <class 'str'>
print(type(True)) # <class 'bool'>
print(type(None)) # <class 'NoneType'>
```

输出格式都是 <class '类型名'>,一目了然。养成用 type() 快速确认类型的习惯,调试时会省很多时间。

2.7 总结与下一步

在这一章,我们认识了 Python 世界最基础的几块积木:

数字:整数 int、浮点数 float,以及它们之间的运算规则。

字符串: str,用引号包裹,支持拼接和 f-string 嵌入。

布尔值: True / False, 来自比较运算, 用于逻辑判断。

None: 表示“空”。

变量: 像标签一样贴在对象上 (详见第 6 章 6.6 节)

type(): 随时查看对象的类型。

这些看起来简单, 但它们是整座大厦的地基。下一章, 我们将学习让代码做判断——if 语句。你会开始写出真正“会思考”的程序。

自检

1. 计算 100 除以 6 的商 (整除) 和余数, 分别打印。
2. 分别用 + 拼接和 f-string 两种方式, 打印一句话: “我今年 25 岁, 身高 175 厘米。” (年龄和身高用变量存储)
3. 试试 `print(10 == "10")`, 结果是 True 还是 False? 为什么?
4. 用 `type()` 查看 3.0、"3.0"、True、None 的类型, 记录输出。
5. 写下你认为 `x = 5` 这行代码在 Python 内部发生了什么? (参考示例: "Python 先创建整数对象 5, 然后把标签 x 贴上去")
6. 建议完成「动手才是真的学」。

第3章 会思考的代码：让程序做判断

上一章我们认识了各种积木：数字、字符串、布尔值。但它们只是躺在那里，什么也不做。想让程序“活”起来，它必须能做判断——根据不同的情况，执行不同的代码。

这一章，我们将学会 if 语句。这是编程的第一个思维拐点：你不再只是写死了的命令清单，而是开始建造岔路口。程序会根据条件，自己选择走哪条路。

3.1 一个真实的问题：我能玩游戏吗？

假设你要写一个程序，用户输入年龄，如果大于等于 18 岁，就显示“可以玩游戏”，否则显示“年龄不够”。

在学 if 之前，我们没法写这种程序。print() 只会傻傻地输出，它不会看条件。

if 语句的思维模型很简单，就是一条岔路：

```
age = int(input("请输入你的年龄："))
if age >= 18:
    print("可以玩游戏")
else:
    print("年龄不够")
```

运行一下，输入 20，屏幕显示“可以玩游戏”；输入 15，显示“年龄不够”。

第一个 if 语句，你已经写出了“会思考”的代码。

3.2 if 语句的结构拆解

把上面那个例子拆开，if 语句由四部分组成：

第一部分：关键字 if

所有判断从这里开始。if 后面跟一个条件——一个布尔值，或者能算出布尔值的表达式。

第二部分：条件

```
age >= 18
```

这就是条件。它运算的结果要么是 True，要么是 False。条件可以是任何能产生布尔值的东西：

```
if True: # 永远执行
if False: # 永远不执行（没意义，只是演示）
```

```
if 10 > 5: # True

if "a" == "b": # False

if age >= 18: # 看 age 的值

if is_logged_in: # 直接用布尔变量
```

新手坑位：条件后面必须加冒号。

忘记冒号是最常见的语法错误。Python 用冒号告诉解释器：“准备好了，下面是一个代码块。”

第三部分：缩进的代码块

冒号下面，缩进的代码就是“条件成立时执行”的内容：

```
if age >= 18:

    print("可以玩游戏") # 缩进了，属于 if

    print("祝你玩得开心") # 也缩进了，也属于 if

print("欢迎光临") # 没缩进，不属于 if，永远执行
```

缩进是 Python 的语法，不是装饰。标准的缩进是 4 个空格。你可以按 Tab 键，VSCode 会自动转换成 4 个空格。不要空格和 Tab 混用，否则会报 `IndentationError`，而且极其难排查。

新手坑位：缩进不一致。

同一个代码块里，缩进必须完全一致。如果第一行缩进 4 格，第二行缩进 3 格或 5 格，Python 就懵了。VSCode 会自动帮你保持缩进，但复制粘贴代码时要格外小心。

第四部分：else（可选）

else 后面也跟冒号和缩进的代码块，在条件不成立时执行。else 不是必须的——如果你只在条件成立时想做什么，不成立时什么都不用干，就可以不写 else。

```
if score >= 60:

    print("及格了") # 不及格的话，什么也不发生
```

3.3 多岔路口：if-elif-else

现实中的判断往往不是非黑即白。比如给分数评等级：

90 分以上：优秀

80~89 分：良好

60~79 分：及格

60 分以下：不及格

elif 就是“否则如果”，可以在中间插入更多条件分支：

```
score = 85
if score >= 90:
    print("优秀")
elif score >= 80:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

执行顺序是从上到下，一个一个条件试。哪个条件先成立，就执行那个代码块，然后直接跳出整个 if 结构，不会再往下试。所以上面的代码输出“良好”，不会再去检查是否大于 60。

新手坑位：条件顺序很重要。

如果你把 `score >= 60` 写在最前面，那么 85 分也会命中“及格”，后面的 elif 就永远没机会了：

```
# 错误的顺序
if score >= 60:
    print("及格") # 85 在这里就匹配了，输出"及格"
elif score >= 80:
    print("良好") # 永远不会被执行，因为 >=60 已经截住了
elif score >= 90:
    print("优秀") # 同上
```

写 if-elif-else 的正确习惯：从最严格的条件开始，逐渐放宽，else 收底。

3.4 条件嵌套：岔路里面还有岔路

判断里面还能再判断。比如电影院：先看有没有票，有票再看年龄够不够：

```
has_ticket = True

age = 16

if has_ticket:
    if age >= 18:
        print("可以观看")
    else:
        print("年龄不够，不能观看")
else:
    print("没有票，不能入场")
```

嵌套是合法的，但不要过度嵌套。如果你发现自己写了三层以上的 if 套 if，代码读起来会像一团乱麻。通常可以用 and 把条件合并，把嵌套“拍平”：

```
if has_ticket and age >= 18:
    print("可以观看")
elif has_ticket and age < 18:
    print("年龄不够，不能观看")
else:
    print("没有票，不能入场")
```

能用 and / or 拍平就拍平，保持代码左对齐，可读性第一。

3.5 条件表达式（进阶选读）

有时候，你只是想在两个值之间选一个，if-else 写四行未免太啰嗦。Python 有一种精简写法，叫条件表达式（也叫三元表达式）：

```
# 啰嗦版

if age >= 18:
    status = "成年"
```

```

else:
    status = "未成年"

# 精简版
status = "成年" if age >= 18 else "未成年"

```

语法是：X if 条件 else Y。条件为真取 X，为假取 Y。

这个很适合用于一行赋值或 f-string 中：

```
print(f"你是{'成年' if age >= 18 else '未成年'}人")
```

初学时不强求，但看到了能认识。

3.6 真值测试：什么条件算是“真”？

if 的条件不只接受 True / False，它还可以接受其他类型的值。Python 会自动把它们判断为“真”或“假”。这个行为叫真值测试。

以下值在 if 语句中被当作 False：

```

False

None

0（任何数值类型的 0：0、0.0、0j）

空的序列或集合：""（空字符串）、[]（空列表）、()（空元组）、{}（空字典）、set()（空集合）

```

剩下的，几乎全都是 True。包括 ""（有空格）、[0]（有内容的列表）、负数等。

这意味着你可以写很简洁的判断：

```

name = input("你的名字: ")

if name:
    print(f"你好, {name}")
else:
    print("你没有输入名字")

```

如果用户直接按回车，name 是空字符串 ""，被判断为 False，触发 else。这种写法在后续处理用户输入、检查数据是否为空时非常常见。

3.7 实战：写一个体重指数计算器（BMI）

这一章开头时我提过，学完判断就能做个小项目。现在来做。BMI 计算公式： $BMI = \text{体重 (kg)} / \text{身高}^2 \text{ (m)}$ 。

```
weight = float(input("请输入体重（公斤）："))
height = float(input("请输入身高（米）："))
bmi = weight / (height ** 2)
print(f"你的 BMI 指数是：{bmi:.1f}")
if bmi < 18.5:
    print("体重过轻")
elif bmi < 24:
    print("正常范围")
elif bmi < 28:
    print("超重")
else:
    print("肥胖")
```

运行，输入 70 和 1.75，输出：

```
你的 BMI 指数是：22.9
正常范围
```

注意这里用了 `{bmi:.1f}`，表示保留 1 位小数显示。这是 `format` 语法的一种，第 2 章没讲，这里混个眼熟。

新手坑位：`input()` 返回的是字符串。

如果你忘了用 `float()` 或 `int()` 转换，`height ** 2` 就会报错，因为字符串不能进行数学运算。这是一个永无止境的坑——只要从 `input()` 拿东西做数学运算，先转换类型。

3.8 总结与下一步

这一章我们赋予了程序“思考”的能力。核心就三样东西：

`if / elif / else`：建造条件岔路。

缩进：Python 用来界定“谁属于谁”的语法。

条件顺序：从上到下，先严格后宽松。

你从“只输出”进化到了“能判断”。但程序现在还是很短，跑完就没了。下一章，我们进入循环——让程序反复做一件事，直到你喊停。你将学会处理待办列表、批量计算，以及如何避免让电脑陷入死循环。

练习

1.用户输入一个整数，判断它是奇数还是偶数。（提示：用 % 取余数，能被 2 整除就是偶数）

2.用户输入年份，判断是不是闰年。规则：能被 4 整除但不能被 100 整除，或者能被 400 整除。

3.用户输入三个数，找出最大的那个并打印。（不用 max()，用 if-else 自己实现。提示：先假设第一个是最大，然后逐个比）

4.if "" 判断为真还是假？if ""（中间有空格）呢？用代码验证并解释结果。

5.优化下面的代码，用 and 把嵌套拍平：

```
score = int(input("分数: "))
if score >= 60:
    if score < 80:
        print("及格")
```

6.建议完成「动手才是真的学」。

第4章 重复的事交给机器：循环的乐趣

上一章我们让程序学会了判断。但程序还有一个绝活：重复做一件事，不厌其烦，速度飞快。

想想这些场景：打印 1 到 100 的所有数字、检查邮件列表里的每一个地址、计算全班 50 个学生的平均分……如果把同一段代码复制粘贴 100 遍，你肯定会疯。这就是循环存在的意义。

这一章，我们学习两种循环：for 和 while。看完你就会写出第一个真正“像程序”的程序。

4.1 一个真实的问题：怎么做 100 个俯卧撑？

如果你想打印 1 到 5，没有循环的时候得这样：

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

那 1 到 100 呢？复制粘贴会死人的。用 for 循环，一句话搞定：

```
for i in range(1, 101):
    print(i)
```

for 循环的思维模型很简单：你有一串东西，从头到尾，挨个拿出来处理一遍。就像老师在教室里点名发糖，点到谁就递给谁一块糖，一次一个，直到全班发完。

4.2 for 循环的结构拆解

```
for i in range(1, 6):
    print(i)
```

结构：

for: 关键字，宣告“我要开始遍历了”。

i: 变量名，每次从序列里取一个值，就贴在它上面。你可以叫它任何名字(n、x、item)，但要有意义。

in: 告诉 Python “从后面的序列里取”。

range(1,6): 生成一个数字序列。range(起点, 终点) 生成从起点到终点-1 的数字 (1, 2, 3, 4, 5)。

冒号: 和缩进的代码块: 和 if 一样, 缩进的部分是循环体, 每次取到一个值就执行一遍。

新手坑位: range(1,6) 不包括 6。

Python 的 range 是“左闭右开”区间, 包含起点, 不包含终点。记住: range(n) 产生 n 个数, 从 0 到 n-1。

range 的几种用法:

```
range(10) # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (给了终点, 起点默认 0)
range(2, 10) # 2, 3, 4, 5, 6, 7, 8, 9 (给了起点和终点)
range(2, 10, 2) # 2, 4, 6, 8 (给了起点、终点、步长)
range(10, 0, -1) # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 (倒着来)
```

4.3 for 不只是遍历数字

for 真正的本事是遍历任何能“拆成一个一个”的东西。比如:

遍历字符串:

```
for char in "Python":
    print(char)
```

遍历列表:

```
fruits = ["苹果", "香蕉", "橘子", "葡萄"]
for fruit in fruits:
    print(f"我喜欢吃{fruit}")
```

列表我们下一章会细讲, 这里你先混个眼熟——用方括号 [] 装起来的东西, for 能挨个拿出来。

4.4 while 循环: 条件不满足就一直跑

for 适合“事先知道有几个东西要处理”的场景。但有时你不知道要跑多少遍, 只知道“只要某个条件成立, 就一直做”。这时候用 while:

```
password = "123456"

guess = ""

while guess != password:

    guess = input("请输入密码: ")

print("密码正确, 欢迎!")
```

while 的思维模型是一个老式打卡机：先检查条件，条件为真就进去跑一圈；回来再检查，还为真就再跑一圈……直到条件为假才离开。

4.5 死循环：一个必须认识的幽灵

while 有一个致命的陷阱：如果条件永远不会变假，程序就跑啊跑，永远不停——这就是死循环。

更典型的死循环是忘了在循环里改变条件变量：

```
# 死循环! x 永远是 1, 条件永远成立

x = 1

while x < 10:

    print(x)

    # 忘了写 x = x + 1
```

如果碰到死循环，在命令行里按 Ctrl + C，可以强行终止。

while 循环的安全三要素：

1. 循环前，初始条件变量要设好。
2. 循环里，条件变量一定要变。
3. 变化的方向迟早让条件变假。

```
count = 0 # 1. 初始化

while count < 5: # 2. 条件

    print(count)

    count += 1 # 3. 变量递进 (+= 是 count = count + 1 的简写)
```

4.6 break 和 continue: 循环里的特权操作

break: 立刻结束整个循环，头也不回。

```
# 猜数字，猜对就退出

answer = 7

while True: # while True 是死循环，但里面有 break 就不怕

    guess = int(input("猜 (1-10): "))

    if guess == answer:

        print("猜对了!")

        break # 退出循环

    print("不对，再试")
```

continue: 跳过本次循环剩余部分，直接进入下一次。

```
# 打印 1-10 里的奇数

for i in range(1, 11):

    if i % 2 == 0: # 偶数

        continue # 跳过，不打印

    print(i)
```

4.7 循环嵌套: 循环里面套循环

和 if 能嵌套一样，循环也能嵌套。最经典的例子就是打印乘法口诀表：

```
for i in range(1, 10):

    for j in range(1, i + 1):

        print(f"{j}×{i}={i*j}", end="\t")

    print() # 换行
```

内层循环完整执行一遍，外层循环才走一步。就像一个老式钟表：秒针走一圈（内层），分针才跳一格（外层）。

4.8 实战：统计班级成绩

给你一个成绩列表，请计算平均分，并找出最高分和最低分。思路：先假设第一个成

绩既是最高也是最低（打擂台），然后遍历逐一比较，发现更高的就更新，发现更低的也更新。

```
scores = [88, 92, 67, 74, 85, 95, 78, 82, 90, 70]

total = 0

max_score = scores[0] # 先假设第一个是最高分
min_score = scores[0] # 先假设第一个是最低分

for score in scores:

    total += score # total = total + score 的简写

    if score > max_score:

        max_score = score

    if score < min_score:

        min_score = score

count = len(scores) # len() 返回列表长度

average = total / count

print(f"平均分: {average:.1f}")

print(f"最高分: {max_score}")

print(f"最低分: {min_score}")
```

4.9 for 和 while 怎么选？

场景	用什么
知道有几个东西，或遍历一个现成的序列	for
不知道要跑几次，靠条件控制	while
无限等用户输入，输入正确才走	while True + break

一个经验法则：能用 for 就不要用 while。for 更安全，不怕死循环。

4.10 总结与下一步

这一章，程序学会了“不厌其烦”：

for：遍历序列，“有一个干一个”。

while：条件控制，“满足条件就一直干”。

break/continue：特殊时候的特权操作。

死循环：`while` 的幽灵，三要素可防可控。

你现在的工具箱：能用变量存东西，能用 `if` 做判断，能用循环批量处理。但要写的程序越来越长，如果类似功能重复出现，你又开始复制粘贴了。下一章，我们学习函数——给一段代码取名字，随叫随到。这是编程的第一个抽象能力。

自检

- 1.用 for 循环打印 1 到 20 的所有偶数。
- 2.用 while 循环实现：从 10 倒数到 1（包括 1），然后打印“发射！”。
- 3.写一个猜数字游戏：程序预设一个 1-100 之间的整数，用户猜，程序提示“大了”或“小了”，直到猜对退出。
- 4.用 for 和 range 生成并打印“九九乘法表”的上三角部分。
- 5.用户连续输入数字，输入空行（直接回车）时结束，然后打印这些数的总和与平均。（提示：input()返回空字符串就是按了回车；用 while True + break）
- 6.建议完成「动手才是真的学」。

第 5 章 组装你的第一套工具：函数

写到第四章，你能做判断，能写循环，能处理列表。你的程序已经能解决一些完整的小问题了。

但你可能会隐约感到不安：代码开始变长，有些逻辑重复出现，你靠复制粘贴改几个数字撑着。如果需求一变，你得改三个地方，漏一个就是 bug。如果程序有两百行，找都找不全。

这就是函数要解决的问题。函数是编程的第一次抽象：给一段代码起个名字，想用的时候喊它一声就行，不必每次把代码再写一遍。

这一章，我们学习如何打造自己的工具库。

5.1 一个真实的问题：为什么复制粘贴三次就该疯了？

拿 BMI 计算说事。上一章我们写了一小段代码，用户输入身高体重，程序弹出指数和评级。现在，你要在一个程序里算三个人的 BMI，然后比较。

没有函数的做法：把那段代码复制三遍，改变量名。代码会变成一坨灾难——又臭又长，想改一下评级的阈值得改三处。

函数的思维是：把“计算 BMI 并返回评级”这件事当成一个黑盒子，你给身高体重，它给你结果。内部怎么算的，调用者不用管。

数据流向示意：

身高, 体重 → [函数：计算 BMI] → (BMI 值, 评级)

用函数之后，主程序变得只有几行，清晰得像目录。

5.2 定义和调用：给一段代码取名字

函数定义的语法：

```
def 函数名():  
    """文档字符串（可选，但强烈建议）"""  
    缩进的代码块
```

调用就是“函数名()”。

看一个最简单的例子：

```
def greet():  
    """打印一句问候语"""  
    print("你好，欢迎！")
```

```
greet()
greet()
```

输出：

你好，欢迎！

你好，欢迎！

`def` 是 “define” 的缩写，告诉 Python：“我要定义一个函数了”。后面是函数名，然后是括号和冒号。缩进的代码是函数体，只有在调用时才会执行。

新手坑位：定义和调用的区别。`def` 这一行只是“登记”了一个函数，并不执行里面的代码。就像你把菜谱写下来，不等于菜做好了。只有写 `greet()` 时，Python 才跑进函数里去执行。

5.3 参数：让函数能接受输入

`greet()` 每次只能打同一句话，太死板。要让函数能对不同的人说不同的话，得给它开个“入口”——这就是参数。

```
def greet(name):
    """向指定的人打招呼"""
    print(f"你好，{name}！")

greet("张三")
greet("李四")
```

输出：

你好，张三！

你好，李四！

括号里的 `name` 叫形参，调用时传进去的“张三”叫实参。调用时，实参被贴上了形参的标签，函数内部就能用这个标签。

用我们第二章的“标签模型”来理解：`greet("张三")` 相当于在函数内部执行了 `name = "张三"`——给字符串对象“张三”贴了一个叫 `name` 的标签。

函数可以有多个参数：

```
def print_info(name, age, city):
    """打印个人信息"""
    print(f"{name}, {age}岁，来自{city}")

print_info("张三", 25, "北京")
```

这行调用等价于在函数内部：

- name = "张三"
- age = 25
- city = "北京"

5.4 返回值：让函数能输出结果

到现在，函数只是“干点啥”（打印）。真正有用的函数往往是“算点啥，把结果交出来”。这就是返回值。

```
def add(a, b):  
    """返回两数之和"""  
    result = a + b  
    return result  
  
sum_result = add(3, 5)  
print(sum_result) # 8  
print(add(10, 20)) # 30
```

return 做两件事：

- 1.立刻结束函数的执行。
- 2.把 return 后面的值交还给调用者。

调用者通常会把返回值赋给一个变量，或者直接嵌入到表达式里。

新手坑位：忘记 return。

如果函数里没有 return，它会默认返回 None。看看这个错误：

```
def add_wrong(a, b):  
    a + b # 算了，但没有 return  
  
print(add_wrong(3, 5)) # None
```

算了是一回事，交出来是另一回事。如果没有 return，计算结果就烂在函数肚子里，外面拿不到。

函数可以返回多个值，实际上是用元组打包：

```
def min_max(numbers):  
    """返回列表的最小值和最大值"""  
    return min(numbers), max(numbers)  
  
low, high = min_max([4, 7, 1, 9])
```

```
print(f"最小: {low}, 最大: {high}")
```

这算不上新语法，只是 `return` 之后跟了几个值，逗号分隔，实际上返回的是一个元组，赋值时自动解包。

5.5 无返回值的函数，其实是返回 `None`

如果你写一个函数只负责打印，不 `return` 任何东西，它也返回了 `None`：

```
def say_hello():  
    print("hello")  
  
result = say_hello()  
print(result) # None
```

这意味着你可以把任何函数的结果赋给变量，但没 `return` 的就是 `None`。在后面学条件判断时，如果误用这种函数返回值会出 `bug`，但现在你心里有数就好。

5.6 文档字符串：写给未来自己看的信

你刚写了一个函数，三天后回来看，可能忘了它是干嘛的，参数什么意思。文档字符串（`docstring`）就是给函数写说明书。

```
def bmi_calc(weight, height):  
    """  
    计算 BMI 指数并返回评级。  
  
    参数:  
        weight: 体重（公斤）  
        height: 身高（米）  
  
    返回:  
        bmi: 计算出的 BMI 数值（浮点数）  
        level: 评级字符串  
    """  
    bmi = weight / (height ** 2)  
    if bmi < 18.5:  
        level = "过轻"  
    elif bmi < 24:  
        level = "正常"  
    elif bmi < 28:  
        level = "超重"
```

```
else:
    level = "肥胖"
return bmi, level
```

三引号包起来的多行字符串就是文档字符串。它放在函数体第一行,是 Python 的约定。你可以用 `help()` 查看它:

```
help(bmi_calc)
```

会打印出你写的说明书。从现在开始,每个函数都写 `docstring`,哪怕只有一行。将来你会感谢自己。

5.7 参数的几种形态

【位置参数】

最常见的按顺序传参:

```
def describe(name, age):
    print(f"{name}, {age}岁")

describe("王五", 30) # name="王五", age=30
```

实参和形参的位置一一对应。这是位置参数。

【关键字参数】

你也可以在调用时指名道姓,不用管顺序:

```
describe(age=30, name="王五") # 效果一样
```

`age=30, name="王五"`就是关键字参数。它们让调用更清晰,尤其参数多的时候。

位置参数和关键字参数可以混用,但位置参数必须在关键字参数前面:

```
describe("王五", age=30) # 正确
describe(name="王五", 30) # 报错! 位置参数不能在关键字参数后面
```

【默认参数】

有时候,大多数调用都用同一个值,只有少数情况例外。你可以给参数设默认值:

```
def greet(name, greeting="你好"):
    print(f"{greeting}, {name}!")

greet("张三") # 你好, 张三!
greet("Tom", greeting="Hello") # Hello, Tom!
```

`greeting="你好"`是默认值。调用时不传,就用默认的;传了就覆盖。

新手坑位（进阶）：默认参数不要用列表或字典。

进阶说明：如果默认值必须是列表，用 `None` 加 `if` 判断来构造新列表。现在先记住“默认参数别用[]”即可。

#危险写法

```
def add_item(item, items=[]):  
    items.append(item)  
    return items
```

#安全写法

```
def add_item(item, items=None):  
    if items is None:  
        items = []  
    items.append(item)  
    return items
```

原因与“标签”和可变对象有关，第6章讲完列表后再回头看会豁然开朗。现在只需记住纪律。

5.8 作用域初探：函数内外是两套世界

在函数里定义的变量，外面看不见：

```
def my_func():  
    x = 10  
    print(x) # 10  
  
my_func()  
print(x) # NameError: name 'x' is not defined
```

函数里的 `x` 是局部变量，只在函数内部有效。函数一结束，它就被回收。

反过来说，函数外面定义的变量，里面能读：

```
y = 20  
def show():  
    print(y) # 20  
  
show()
```

但如果想在函数里修改外面的变量，得用 `global` 关键字声明，否则 Python 会认为你定义了一个新的局部变量：


```

counter = 0
def increment():
    global counter
    counter += 1

increment()
print(counter) # 1

```

我的建议：初学阶段，尽量避免用 `global`。用参数把值传进去，用 `return` 把结果传出来。这样函数是独立的，调试容易，逻辑清晰。`global` 会让数据流向变得杂乱，只在很特殊的情况下用。

5.9 实战：把 BMI 计算器封装成工具函数

我们之前写的 BMI 代码混在一起，现在把它重构为函数：

```

def calculate_bmi(weight, height):
    """
    计算 BMI 指数。

    参数：
        weight: 体重（公斤）
        height: 身高（米）

    返回：
        bmi 数值（浮点数）
    """
    return weight / (height ** 2)

def get_bmi_level(bmi):
    """
    根据 BMI 数值返回评级。

    参数：
        bmi: BMI 数值

    返回：
        评级字符串
    """
    if bmi < 18.5:
        return "过轻"
    elif bmi < 24:
        return "正常"

```

```

    elif bmi < 28:
        return "超重"
    else:
        return "肥胖"

def print_bmi_report(name, weight, height):
    """打印一份 BMI 报告"""
    bmi = calculate_bmi(weight, height)
    level = get_bmi_level(bmi)
    print(f"{name}: BMI {bmi:.1f}, {level}")

#主程序

print_bmi_report("张三", 70, 1.75)
print_bmi_report("李四", 85, 1.80)
print_bmi_report("王五", 55, 1.65)

```

输出：

张三：BMI 22.9，正常

李四：BMI 26.2，超重

王五：BMI 20.2，正常

这个结构有几个好处：

- 1.单一职责：每个函数只做一件事。calculate_bmi 只管算数，不管评级。get_bmi_level 只管评级，不管打印。
- 2.可测试：你可以单独调用 calculate_bmi 检查计算结果，不用跑整个报告。
- 3.可复用：以后任何地方要用 BMI 评级，直接 import 这个模块就行（第 10 章细说），不用再写一遍。

5.10 总结与下一步

这一章，我们学会了把代码装进盒子里，贴上标签。

def 定义函数，函数名()调用。

参数给函数开入口（位置、关键字、默认值）。

return 给函数开出口。

文档字符串是写给人看的说明书。

作用域告诉我们函数内外的变量互不干扰。

函数封装让代码模块化、可复用、好维护。

从现在开始，你写的每一个稍大的程序，都应该想：能不能拆成函数？每个函数只做一件事，名字写清楚，别人（包括未来的你自己）一看就懂。

下一章，我们要深入 Python 最常用的数据结构——列表、元组、字典、集合。它们是你处理批量数据的利器。同时，你对变量的“标签模型”将在那里得到完整验证。

自检

1. 写一个函数 `is_even(n)`，判断一个整数是否为偶数，是返回 `True`，否返回 `False`。然后用它打印 1 到 20 的所有偶数。

2. 写一个函数 `max_of_three(a, b, c)`，返回三个数中的最大值。不要用内置的 `max()` 函数。

3. 写一个函数 `print_multiplication_table(n)`，打印 `n` 的乘法口诀（ $1 \times n$ 到 $9 \times n$ ）。用 `for` 循环实现。

4. 写一个函数 `count_vowels(s)`，统计字符串 `s` 中元音字母（`a, e, i, o, u`）的个数（忽略大小写），返回个数。（提示：先用 `s.lower()` 转小写再统计）

5. 把上一章的猜数字游戏改写成函数版本：定义 `guess_number(answer)`，在函数内部实现用户交互，直到猜对退出。主程序只写 `guess_number(42)` 即可运行。

6. 建议完成「动手才是真的学」。

第6章 数据的家：列表、元组、字典、集合深入

你已经能写函数，能处理少数几个变量。但现实中的数据往往是一堆一堆来的——全班同学的成绩、购物车里的商品、通讯录里的联系人。你需要能装很多个东西的容器。

Python 给这类“批量数据”准备了四套容器：列表、元组、字典、集合。它们各有脾气，选对容器能让代码又短又快，选错则平白多出很多麻烦。

这一章我们把这四样东西讲深讲透。你甚至会自己实现一个简单通讯录。

6.1 列表：最自由的序列

列表是最常用的容器。它用方括号[]包裹，里面的元素用逗号隔开：

```
numbers = [1, 2, 3, 4, 5]
fruits = ["苹果", "香蕉", "橘子"]
mixed = [1, "hello", True, None] # 可以混装不同类型
```

6.1.1 索引和切片：精准定位

列表是有序的。每个元素都有一个编号——索引，从0开始。

```
letters = ["a", "b", "c", "d", "e"]
print(letters[0]) # a
print(letters[2]) # c
print(letters[-1]) # e （负数从右往左数，-1 是最后一个）
```

新手坑位：索引从0开始，不是1。几乎所有编程语言都这样，因为计算机内部偏移量就是从0算起的。如果你取 letters[5]，会报 IndexError: list index out of range，因为最大索引是4。

切片能取出一段子列表，语法是[起点:终点:步长]，和 range 一样左闭右开：

```
print(letters[1:4]) # ['b', 'c', 'd'] （索引 1 到 3）
print(letters[:3]) # ['a', 'b', 'c'] （省略起点，默认从 0）
print(letters[2:]) # ['c', 'd', 'e'] （省略终点，默认到末尾）
print(letters[::2]) # ['a', 'c', 'e'] （步长 2）
print(letters[::-1]) # ['e', 'd', 'c', 'b', 'a'] （反转）
```

切片不会修改原列表，而是生成一个新列表。

6.1.2 修改、增加、删除

列表是可变的，你可以动它的内部元素：

```
nums = [10, 20, 30]
nums[1] = 25 # 直接赋值修改
print(nums) # [10, 25, 30]
```

增加元素的方法：

```
nums.append(40) # 尾部追加一个
nums.insert(1, 15) # 在索引 1 处插入 15
nums.extend([50,60]) # 尾部扩展多个元素
```

删除元素：

```
del nums[2] # 按索引删
nums.remove(30) # 按值删（删第一个找到的）
popped = nums.pop() # 弹出最后一个，返回值给你
popped2 = nums.pop(0) # 弹出索引 0 的元素
nums.clear() # 清空
```

新手坑位：remove 只删第一个匹配值，如果值不存在会报错。删之前最好用 in 检查一下，或者用 try/except 兜底（第 7 章会讲）。

6.1.3 列表推导式：一行生成列表

如果你想生成一个 1 到 10 的平方列表，可能会这样写：

```
squares = []
for i in range(1, 11):
    squares.append(i ** 2)
```

Python 提供了一种更简洁的写法——列表推导式：

```
squares = [i ** 2 for i in range(1, 11)]
```

语法：[表达式 for 变量 in 可迭代对象]，还能加 if 过滤：

```
even_squares = [i ** 2 for i in range(1, 11) if i % 2 == 0]
#[4, 16, 36, 64, 100]
```

列表推导式执行快，表达力强，但不要让它变得太复杂——如果逻辑超过一层循环加一层判断，拆成普通循环反而更可读。

6.2 元组：不可变的序列

元组和列表长得几乎一样，只是用圆括号()而非方括号：

```
point = (3, 4)
rgb = (255, 128, 0)
```

核心区别：元组不可变。一旦创建，不能修改、不能增加、不能删除。下面的操作全都会报错：

```
point[0] = 10 # TypeError: 'tuple' object does not support item assignment
```

那为什么要用它？两个好处：

1.安全性：你不希望某个数据在传递过程中被意外篡改时，就用元组。比如函数的配置参数、坐标点。

2.性能：元组比列表轻量，处理更快。如果你有一堆不需要改的数据，用元组更省内存。

新手坑位：单元素元组必须加逗号。

(42)被 Python 解析为括号里的整数 42，而不是元组。要写成(42,)才算只有一个元素的元组。这个小细节经常让人莫名其妙。

元组支持索引和切片，用法和列表一样，只是不能修改。

【元组解包】

非常方便的一个特性：

```
point = (3, 4)
x, y = point
print(x) # 3
print(y) # 4
```

这个技巧也适用于列表：

```
colors = ["red", "green", "blue"]
r, g, b = colors
```

交换两个变量值，不用中间变量：

```
a, b = b, a
```

这背后其实是元组解包的功劳：右边 b, a 构成了一个临时元组，然后解包给 a, b。

6.3 字典：现实世界的映射表

列表和元组按顺序装数据。但生活中更常见的是名字对应值：人名对应电话号码，商品名对应价格，水果对应库存数量。字典就是做这个的。

字典用花括号{}, 里面是键:值对:

```
person = {  
    "name": "张三",  
    "age": 25,  
    "city": "北京"  
}
```

键通常是字符串, 值可以是任何对象。通过键来获取值, 而不是索引:

```
print(person["name"]) # 张三  
print(person["age"]) # 25
```

新手坑位: 访问不存在的键会报 `KeyError`。如果你不确定键在不在, 用 `get` 方法更安全:

```
print(person.get("phone", "N/A")) # N/A, 不存在时返回默认值
```

【字典的增删改查】

```
person["phone"] = "123456" # 新增键值对  
person["age"] = 26 # 修改  
del person["city"] # 删除  
print(person.keys()) # dict_keys(['name', 'age', 'phone'])  
print(person.values()) # dict_values(['张三', 26, '123456'])  
print(person.items())
```

遍历字典:

```
for key, value in person.items():  
    print(f"{key}: {value}")
```

【字典推导式】

和列表推导式类似, 可以快速生成字典:

```
squares = {x: x**2 for x in range(1, 6)}  
#{1:1, 2:4, 3:9, 4:16, 5:25}
```

6.4 集合: 不重复的无序容器

集合是数学里“集合”的体现: 没有重复元素, 没有顺序。用花括号或 `set()` 创建:

```
fruits = {"苹果", "香蕉", "橘子", "苹果"}
print(fruits) # {'橘子', '香蕉', '苹果'} (不重复, 顺序不定)
```

【集合的核心用途：去重和成员检查】

去重非常简单：

```
nums = [1, 2, 2, 3, 3, 3]
unique_nums = list(set(nums)) # [1, 2, 3]
```

检查一个元素是否存在，集合比列表快得多（内部是哈希表）：

```
if "苹果" in fruits:
    print("有苹果")
```

【集合运算】

你可以做数学里的集合运算：交集&，并集|，差集-，对称差集^：

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a & b) # {3, 4}
print(a | b) # {1, 2, 3, 4, 5, 6}
print(a - b) # {1, 2}
print(a ^ b) # {1, 2, 5, 6}
```

集合推导式：

```
s = {x ** 2 for x in range(5)} # {0, 1, 4, 9, 16}
```

新手坑位：空花括号{}是字典，不是集合。创建空集合只能用 set()。

6.5 四种容器对比与选择

列表(list)：有序，可变，允许重复。主要用途：顺序存储、遍历。

元组(tuple)：有序，不可变，允许重复。主要用途：安全传递、坐标。

字典(dict)：Python 3.7+保持插入序，可变，键不允许重复（值可）。主要用途：名称到值的映射。

集合(set)：无序，可变，不允许重复。主要用途：去重、集合运算。

选择建议：

- 1.需要顺序存储，数据可能变 → 列表
- 2.需要保护数据不被改，或作为字典键 → 元组（不可变才能当键）
- 3.通过名字访问数据 → 字典
- 4.去重、检查存在、求交集 → 集合

6.6 深入理解：标签模型与可变对象

第二章我们强调变量是标签，不是盒子。现在有了列表，这个区别终于要显露威力了。

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
```

为什么改了 b, a 也变了？因为 a 和 b 两个标签都贴在同一个列表对象上。b.append(4) 是让那个列表对象增加了元素，两个标签都指向这个已经变化了的对象。

如果把 b 重新绑定到新列表，a 不受影响：

```
b = [5, 6, 7]
print(a) # [1, 2, 3, 4]
```

新手坑位（进阶）：默认参数不要用列表或字典作为默认值。

6.7 实战：写一个简单通讯录

要求：可以用名字添加电话号码，可以查询，可以列出所有联系人。

```
def add_contact(contacts, name, phone):
    """添加或更新联系人"""
    contacts[name] = phone
    print(f"已保存: {name} -> {phone}")

def find_contact(contacts, name):
    """查询联系人，返回电话号码或提示"""
    return contacts.get(name, "未找到该联系人")

def show_all(contacts):
    """显示全部联系人"""
    if not contacts:
        print("通讯录为空")
    else:
        for name, phone in contacts.items():
            print(f"{name}: {phone}")

#主程序

phonebook = {}
while True:
```

```

action = input("操作: 添加(1)/查询(2)/全部(3)/退出(0): ")
if action == "1":
    name = input("姓名: ")
    phone = input("电话: ")
    add_contact(phonebook, name, phone)
elif action == "2":
    name = input("姓名: ")
    print(find_contact(phonebook, name))
elif action == "3":
    show_all(phonebook)
elif action == "0":
    break
else:
    print("无效操作")

```

6.8 总结与下一步

这一章你掌握了 Python 的四大容器：列表、元组、字典、集合。它们是你数据处理的基石。

列表：有序可变，最常用。

元组：有序不可变，安全高效。

字典：键值映射，查找飞快。

集合：无序无重复，运算强大。

下一章，我们将学习文件和异常处理。你的程序不再只是运行完就消失，而是能读写真实文件，保存数据，并且优雅地应对各种意外情况。

自检

1. 创建一个列表，包含数字 1 到 10，用推导式生成它们的立方组成的列表。
2. 有一个列表 `nums = [3, 5, 2, 8, 2, 5, 1, 3]`，用集合去重并保留唯一元素。
3. 创建一个字典，键为 "apple"、"banana"、"cherry"，值为对应的库存数量，然后依次打印每种水果的库存。
4. 写一个函数 `common_elements(list1, list2)`，返回两个列表的公共元素（不重复），用集合实现。（提示：`set(list1) & set(list2)`）

第7章 文件与异常：跟外部世界对话

你写的程序已经能计算、判断、循环、处理列表和字典。但每次运行结束，所有数据都消失得无影无踪。你肯定想要保存结果——把数据写进文件，下次运行时再读出来。

同时，程序不可能总在理想环境下运行。文件可能不存在，用户可能输入奇怪的东西，网络可能断开。如果程序一碰到意外就崩溃，那它永远谈不上“能用”。

这一章，我们学习两样让程序从“玩具”变成“工具”的本事：文件读写和异常处理。学完你能写出能保存数据的程序，而且它不会动不动就炸。

7.1 为什么需要文件和异常？

没有文件能力的数据处理，就像用粉笔在黑板上演算——一擦就没。有了文件，你的程序可以把结果存成.txt、.csv，下次启动再读回来。日志文件、配置文件、用户数据，几乎一切持久化都靠文件。

没有异常处理的程序，就像走路不看脚下。用户输错一个字符，或者文件被移动了位置，整个程序立刻崩溃，甩一堆红字给用户。异常处理就是给程序铺安全网：出事了能兜住，给出友好提示，甚至可以尝试恢复。

7.2 打开文件：建立跟外部世界的通道

Python 打开文件用 `open()` 函数，它返回一个文件对象，你可以通过这个对象读或写。

```
f = open("test.txt", "r") # 以只读模式打开
```

"r"是模式（mode），常见的有：

"r"：只读。文件不存在时报错。

"w"：只写（清空已有内容）。文件不存在时创建新文件。

"a"：追加（从末尾接着写）。文件不存在时创建新文件。

"r+"：读写。文件不存在时报错。

默认就是"r"，所以 `open("test.txt")` 和 `open("test.txt", "r")` 一样。

新手坑位：路径问题。

如果只给文件名，Python 在当前工作目录找。VSCode 的当前目录是终端打开的文件夹。文件找不到就报 `FileNotFoundError`。建议先用绝对路径或确保文件在正确位置。你也可以用 `os.getcwd()` 查看当前目录。

打开后，记得关闭文件：

```
f.close()
```

不关的话，可能造成数据没完全写入、系统资源占用等奇怪问题。但手动关闭很容易忘，所以 Python 提供了更安全的 with 语句。

7.3 with 语句：自动关门的管家

with 语句会在代码块执行完自动关闭文件，即使中间出了异常也能正确关闭：

```
with open("test.txt", "r") as f:
    content = f.read()
    print(content)
```

#出了这个缩进，文件已经关闭了

从今往后，只使用 with 来打开文件。习惯成自然，你不再需要操心 close()。

7.4 读文件：把文件内容变成字符串或列表

文件对象提供几种读取方法。

read() 一次性全读

```
with open("test.txt", "r") as f:
    content = f.read()
    print(content)
```

read()把整个文件读成一个字符串，适合小文件。如果文件有几个 GB，猛一下全读进内存可能会炸，这时候需要一行一行读。

readline() 读一行

每次调用读一行，返回的字符串包括行末的换行符\n：

```
with open("test.txt", "r") as f:
    first = f.readline()
    second = f.readline()
    print(repr(first)) # repr 能看到 \n 等转义字符
```

readlines() 全读成列表

每一行作为列表的一个元素：

```
with open("test.txt", "r") as f:
    lines = f.readlines()
print(lines) # ['第一行\n', '第二行\n', '第三行\n']
```

直接迭代文件对象（最常用）

文件对象本身就是可迭代的，for 循环自动逐行读取，内存占用小：

```
with open("test.txt", "r") as f:
    for line in f:
        print(line.strip()) # strip() 去掉行末的换行符和空白
```

这是处理大文件的推荐方式。

7.5 写文件：把程序的成果存下来

模式"w"或"a"打开后，用 write()或 print()写入。

#写模式：会覆盖已有内容

```
with open("output.txt", "w") as f:
    f.write("第一行\n")
    f.write("第二行\n")
```

如果要保留原有内容并在后面追加，用"a"模式：

```
with open("output.txt", "a") as f:
    f.write("这是追加的一行\n")
```

print()也可以写到文件，用 file 参数：

```
with open("output.txt", "w") as f:
    print("用 print 写文件", file=f)
```

新手坑位："w"会清空原有数据。

如果不小心对一个重要文件用了"w"而不是"a"，数据就丢了。写之前一定要确认模式对不对。

7.6 异常处理：给程序上保险

文件找到没？用户输入的是数字吗？网络还在吗？这些问题用 if 判断往往不够，因为错误可能在很深的底层发生。Python 的异常机制提供了统一处理方案。

【基本语法：try-except】

```
try:
    num = int(input("请输入数字: "))
    print(f"你输入了 {num}")
except ValueError:
    print("那不是数字!")
```

如果 try 块里的代码抛出 ValueError(比如输入"abc"), 程序不会崩溃, 而是跳到 except 块执行。

一个 try 可以对应多个 except, 处理不同异常类型:

```
try:
    x = int(input("第一个数: "))
    y = int(input("第二个数: "))
    print(x / y)
except ValueError:
    print("请输入整数")
except ZeroDivisionError:
    print("不能除以零")
```

【抓住异常对象】

有时候你想知道具体错误信息:

```
try:
    with open("不存在.txt", "r") as f:
        content = f.read()
except FileNotFoundError as e:
    print(f"文件没找到: {e}")
```

e 是异常对象, 包含错误描述。

【finally: 无论如何都执行】

finally 块里的代码, 无论是否发生异常, 是否 return, 都会执行。常用来做清理工作:

```
f = None
try:
    f = open("test.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("文件不存在")
finally:
    if f:
        f.close()
```

不过, 如果你用 with, 连 finally 都省了——with 会自动调用关闭。

【何时用异常, 何时用 if?】

经验法则:

- 1.如果错误可预期且能直接检查，用 if。比如 if s == "" 检查空输入。
- 2.如果错误不可预期、涉及外部资源、操作是否成功需要实际尝试，用 try-except。比如打开文件、网络请求、数据库写入。

7.7 实战：统计小说词频

我们来一个完整实战：读入一本小说文本文件，统计每个单词出现的次数，输出前 20 个高频词。

假设我们有一个 novel.txt，内容是英文小说。我们要把所有单词转小写，用空格和标点分割，排除掉 the、a、is 等停用词。

```
import string

def count_words(filename):
    """从文件读取文本，统计词频，返回字典 {单词:次数}"""

    word_counts = {}

    try:
        with open(filename, "r", encoding="utf-8") as f:
            for line in f:
                line = line.lower()
                line = line.translate(str.maketrans("", "", string.punctuation))
                words = line.split()
                for word in words:
                    word_counts[word] = word_counts.get(word, 0) + 1

    except FileNotFoundError:
        print(f"文件 {filename} 不存在")
        return {}

    return word_counts

def top_words(word_counts, n=20):
    """返回出现次数最多的 n 个词"""

    sorted_items = sorted(word_counts.items(), key=lambda x: x[1], reverse=True)
```

```

    return sorted_items[:n]

if __name__ == "__main__":

    filename = "novel.txt"

    counts = count_words(filename)

    if counts:

        top = top_words(counts, 20)

        for word, freq in top:

            print(f"{word}: {freq}")

```

使用示例（如果你有 novel.txt）

```

filename = "novel.txt"

counts = count_words(filename)

if counts:

    top = top_words(counts, 20)

    for word, freq in top:

        print(f"{word}: {freq}")

```

代码解读：

string.punctuation 是标点符号集合。

maketrans 和 translate 配合用于删除标点。

word_counts.get(word, 0) + 1 是字典计数惯用法。

按出现次数排序这个操作你现在只需要知道它能用，具体语法（lambda）你可以自行学习。文件用 encoding="utf-8" 明确编码，避免中文乱码。

新手坑位：编码问题。

Windows 上文本文件有时是 GBK 编码，直接 utf-8 打开会报 UnicodeDecodeError。如果你遇到，可以尝试 encoding="gbk" 或 errors="ignore"。最好保存文件时统一用 UTF-8 编码。

7.8 常见异常清单

你迟早会碰到这些，先认一认：

FileNotFoundError: 文件不存在

ValueError: 类型转换失败, 如 `int("abc")`
ZeroDivisionError: 除以零
TypeError: 对不支持的类型做操作, 如 `"a" + 5`
IndexError: 列表索引超出范围
KeyError: 字典键不存在
NameError: 使用未定义的变量
SyntaxError: 代码语法错误 (运行前就报)
IndentationError: 缩进错误

SyntaxError 和 IndentationError 是程序跑之前就报的, 不能用 `try-except` 捕获, 需要你
自己检查代码。

7.9 总结与下一步

这一章, 你的程序首次迈出自己的一亩三分地, 跟外部文件打交道:

`open()`和模式 `r/w/a`: 建立通道。

`with` 语句: 让文件安全自动关闭。

读: `read()`、`readline()`、直接迭代。

写: `write()`和 `print(..., file=...)`。

异常处理 `try-except-finally`: 兜住意外, 优雅崩溃或恢复。

实战: 用字典做词频统计, 这是很多文本分析的基础。

你的程序现在能记住数据, 也能抗住失误。下一章, 我们将学习面向对象编程——把
数据和操作封装成类和对象。这是 Python 最重要的组织思想, 你之前用的字符串、列表,
本质上全都是对象。我们要开始自己设计对象了。

自检

1. 写一个程序, 让用户输入多行文字 (空行结束), 保存到 `notes.txt`, 然后读出来显示。
2. 修改上面的程序, 如果 `notes.txt` 已有内容, 新输入的内容追加到末尾, 而不是覆盖。
3. 写一个函数 `safe_divide(a, b)`, 返回 `a/b` 的结果。如果 `b` 是 0 或者传入的不是数字, 捕获异常并返回 `None`。
4. 读取一个文本文件 (自己创建), 统计总行数、总单词数、总字符数 (类似 `wc` 命令)。(提示: 逐行读取, `len(line)`统计字符数, `line.split()`统计单词数)
5. 用 `try-except` 读取一个不存在的文件, 捕获 `FileNotFoundError` 并打印友好提示。
6. 建议完成「动手才是真的学」。

附录 A GitHub 共享指南：把你的代码放到全世界的舞台上

你写的代码，如果不分享出去，就像一本锁在抽屉里的日记——只有自己知道它有多精彩。GitHub 是全世界最大的代码托管平台，它不只能存代码，更是一个协作平台和个人名片。你把代码推上去，别人就能看到、使用、甚至帮你改进。找工作时，一个活跃的 GitHub 主页胜过千言万语。

这份指南假设你从未用过 Git 或 GitHub。我们从零开始，把最核心的操作串一遍：从安装 Git，到把你的项目推上 GitHub，再到从别人那里拉取更新。全程跟着做，半小时你就能拥有自己的第一个开源仓库。

A.1 Git 和 GitHub：一个是工具，一个是平台

很多人把 Git 和 GitHub 混为一谈，但它们不是一回事。

Git 是一个版本控制系统。它安装在你的电脑上，负责跟踪文件的每一次修改。你可以随时回退到三天前的版本，可以同时开多条开发线而互不干扰。

GitHub 是一个网站，提供 Git 仓库的远程托管服务。你把本地 Git 仓库的“快照”推送（push）到 GitHub 上，别人就能看到和下载。

简单说：Git 是本地版本管理工具，GitHub 是远程协作平台。类似的平台还有 GitLab、Gitee。

A.2 安装 Git

去 git-scm.com 下载安装包。安装过程中一路默认即可。有一个选项 “Git Bash Here” 非常有用，它会给你一个模拟 Linux 风格命令行——本书后面命令都在 Git Bash 里执行。

安装完成后，在任意文件夹空白处右键，选择 “Git Bash Here”，会弹出一个终端窗口。输入：

```
git --version
```

看到类似 `git version 2.44.0` 就成功了。

A.3 告诉 Git 你是谁

每个 Git 提交都会记录提交者信息。先做一次性全局配置：

```
git config --global user.name "你的名字" git config --global user.email "你的邮箱"
```

这里的名字和邮箱会作为你的“电子签名”，出现在每一次提交里。建议用注册 GitHub 时用的邮箱。

A.4 创建 GitHub 仓库

打开 github.com 注册并登录。(打不开网站？请看 A.4.1)

点击右上角头像旁边的 “+” → “New repository”。

仓库名 (Repository name) 填 `my-first-project` (或用你实际项目名)。

描述 (Description) 可选填。

选择 Public (公开) 或 Private (私人)。初学者建议 Public，让大家看到你的作品。

不要勾选 “Add a README file” 等初始化选项——我们要从本地推上去一个已有项目，勾选了反而会冲突。

点击 “Create repository”。

你会跳转到一个空仓库页面，上面已经贴心地给出了“推送现有仓库”的指令。先别急，我们回到本地操作。

A.4.1 无法正常访问 Github

国内连不上 Github 太正常了。

方法一：使用 Watt Toolkit 加速器

前往 <https://steampip.net/> 下载，推荐用蓝奏云分流

安装打开选中 Github 加速即可。

这个方法最简单

方法二：代理

这个不让说。

A.5 把本地项目变成 Git 仓库

假设你有一个项目文件夹叫 `my-first-project`，里面有一个 `main.py`。

打开 Git Bash (或终端)，进入这个文件夹：

```
cd my-first-project
```

初始化 Git 仓库：

```
git init
```

这个命令在文件夹里创建了一个隐藏的 .git 目录，它记录所有版本历史。现在这个文件夹已经是一个 Git 仓库了。

查看状态

```
git status
```

你会看到类似输出：

```
On branch main
No commits yet
Untracked files:
  main.py
```

Git 发现了一个 main.py，但它还没有被跟踪（untracked）。我们要告诉 Git：“这个文件归你管了”。

添加文件到暂存区

```
git add main.py
```

或者更简单，添加当前目录下所有文件：

```
git add .
```

git add 把文件放进暂存区（staging area）——相当于你把东西打包好，准备签个字正式归档。

提交（commit）：签上你的名字

```
git commit -m "第一次提交：添加主程序"
```

-m 后面跟提交信息（commit message）。这是一个简短描述，告诉别人（和未来的自己）这次改动做了什么。务必认真写提交信息——它是代码的“日志”，关键时刻能救命。

现在，你的改动已经正式记录在本地的 Git 历史里了。

A.6 把本地仓库推到 GitHub（push）

回到刚才的 GitHub 空仓库页面。你会看到一行命令：

```
git remote add origin https://github.com/你的用户名/my-first-project.git
```

复制它，回到 Git Bash 粘贴执行。这条命令告诉本地仓库：“我的远程老家叫 origin，地址是那个网址”。

接下来，第一次推送：

```
git branch -M main
```

```
git push -u origin main
```

`git branch -M main`: 确保本地主分支叫 `main` (GitHub 的新仓库默认主分支名也是 `main`)。

`git push -u origin main`: 把本地 `main` 分支推送到远程 `origin`, `-u` 把本地分支和远程分支关联起来, 以后直接 `git push` 就行。

如果弹出登录窗口, 按提示用浏览器授权 GitHub 登录即可。推送成功后, 刷新 GitHub 页面, 你的代码已经出现在哪里了!

新手坑位: 认证方式。

2021 年起, GitHub 不再接受纯密码认证。推荐两种方式:

个人访问令牌 (Token): 在 GitHub Settings → Developer settings → Personal access tokens 生成一个 token, 权限勾选 `repo`。之后当 Git 提示输入密码时, 粘贴这个 token。

SSH 密钥 (推荐): 生成 SSH 密钥对, 把公钥传到 GitHub。后续无需每次输密码。GitHub 官方文档有详细步骤, 搜索 “GitHub SSH key”。

A.7 日常修改流程: 改代码 → `add` → `commit` → `push`

以后你每次修改代码并想要保存到 GitHub, 只需三步:

```
git add .      # 暂存所有改动
git commit -m "修复了登录按钮的样式"  # 提交
git push       # 推到 GitHub (已关联, 无需加参数)
```

养成小步提交的习惯——每完成一个小功能或修复一个 `bug` 就提交一次。不要攒了三天代码才一次性提交, 那样一旦出问题很难回溯。

A.8 从 GitHub 拉取更新 (`pull`)

如果你的项目被其他协作者修改了, 或者你自己在另一台电脑上推送了代码, 现在想在本地同步最新版本, 就需要拉取。

```
git pull origin main
```

如果是第一次从 GitHub 下载别人的项目到本地, 没有本地仓库, 用 `clone`:

```
git clone https://github.com/用户名/仓库名.git
```

这会把整个仓库下载到当前目录, 自动建立好本地与远程的关联。

A.9 常用的周边操作

查看提交历史

```
git log --oneline
```

每一行是一个提交，前面是提交哈希（唯一 ID），后面是提交信息。按 q 退出。

放弃本地修改（未 add 的）

如果你改乱了某个文件，想回到上次提交时的状态：

```
git checkout -- 文件名
```

撤销 add（已暂存但未提交）

```
git reset HEAD 文件名
```

添加 .gitignore 文件

有些文件不应该入库——比如 __pycache__、虚拟环境 venv、系统临时文件。在仓库根目录创建一个 .gitignore 文件（注意文件名就是 .gitignore），内容按行写要忽略的模式：

```
__pycache__/venv/*.pyc.DS_Store
```

添加 .gitignore 后也要 git add .gitignore 并提交，它才会生效。

新手坑位：密钥和密码绝对不要提交到 GitHub。

像 password = "123456" 或 API 密钥，一旦 push 上去，即使你删掉再提交，历史记录里仍然能找出来。用环境变量或配置文件管理敏感信息，并把配置文件加入 .gitignore。

A.10 分支简介：同时开发多个功能不打架

分支是 Git 真正的灵魂。它让你从主线代码中分出一条“岔路”，在上面试验新功能，不影响主线。等新功能稳定了，再合并回主线。

创建并切换到新分支：

```
git checkout -b new-feature
```

现在你在 new-feature 分支。写代码、add、commit，都和 main 分支互不影响。要切回 main：

```
git checkout main
```

等新功能完成，合并回 main：

```
git merge new-feature
```

合并完如果不再需要那个分支，可以删除：

```
git branch -d new-feature
```

对于个人小项目，你可以一直只在 main 上开发。但一旦开始协作，或者想保持一个“永远能跑”的稳定版本，分支就是必需品。

A.11 总结

这一套流程就是 Git 的日常：

本地初始化：git init, git add, git commit

关联远程：git remote add origin 地址

推送：git push

拉取：git pull

克隆：git clone

从这个指南出发，你已经能把任何项目发布到 GitHub 上。当你开始参与开源、和别人共同维护代码时，你会慢慢接触更高级的操作（变基 rebase、解决冲突、Pull Request）。但这一切的根基，正是你现在掌握的 push 与 pull。

现在，挑一个你前面写的 Python 项目，按这个指南推上 GitHub，然后把仓库链接发给朋友看一看。代码只有被看见，才真正拥有了生命。

动手才是真的学

使用规则：

1. 先猜再跑——每次运行前，写出你预期的输出
2. 每个实验至少做一次"故意搞坏"——改数字、删括号、少写冒号，看报错信息
3. 想 10 分钟没思路再看提示，不要瞄一眼题目就翻答案
4. 实验 3.3（真值测试）和实验 6.1（列表变异）是必做，其他自选

第 2 章实验：认积木

实验 2.1：数字实验室

打开交互模式（命令行输 `python`），依次输入：

```
print(10 / 3)
print(10 // 3)
print(10 % 3)
print(10 ** 3)
```

破坏挑战：试试 `10/0`，记住这个错误长相，以后会一直见到它。

实验 2.2：字符串拼装

```
a = "Python"
b = "很有趣"
print(a + b)
print(a * 3)
```

破坏挑战：试试 `a + 3`（不加 `str()`），再试试 `a * "2"`，看看分别报什么错。

实验 2.3：f-string 对比

```
name = "你"
print(f"欢迎{name}，今天{2026 - 2000}年")
```

改成你的名字和出生年份，看看结果。

实验 2.4: type 侦探

```
print(type(42))
print(type(42.0))
print(type("42"))
print(type(True))
print(type(False))
print(type("True"))
```

注意 "42" 和 42 的区别, True 和 "True" 的区别。

第 3 章实验: 造岔路

实验 3.1: 温度转换

写一个程序, 用户输入摄氏度, 输出华氏度。公式: $F = C \times 9/5 + 32$ 。再加一句判断: 如果温度低于 0°C , 打印"结冰了"。

实验 3.2: 登录模拟

补全下面代码: 密码正确打印"欢迎", 错误打印"密码错误", 连续错 3 次打印"账户锁定"。

```
stored_password = "python123"
user_input = input("请输入密码: ")
```

提示: 需要一个计数器。

实验 3.3: 真值测试 (必做)

创建一个列表, 写循环判断每个元素的真假, 打印出"真"或"假"。

```
tests = [0, 1, "", " ", [], [0], None, False, True]
for item in tests:
# 你的代码
```

破坏挑战: 猜猜 if "False": 是真是假? 先猜再验证。

第 4 章实验：让机器重复

实验 4.1：画三角

用循环打印出：

```
*  
**  
***  
****  
*****
```

提示：`range(1,6)` 和字符串乘法 `*`。

扩展：改成用户输入行数，打印出对应高度的三角形。

实验 4.2：猜数字——你自己的版本

不用书上的代码，自己从头写一个猜数字游戏。规则：程序随机生成 1-100 的数，用户猜，提示“大了/小了”，猜对后显示猜了多少次。

```
import random  
  
answer = random.randint(1, 100)
```

破坏挑战：如果用户输入“abc”，程序会崩溃。试试看，然后思考怎么处理。

实验 4.3：百元百鸡

公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元三只。用 100 元买 100 只鸡，有几种买法？

提示：三重循环，或者两重循环加数学计算。

第 5 章实验：造工具

实验 5.1：拆弹改造

找一章之前写过的代码（比如 BMI 计算器或猜数字），把它拆成至少 3 个函数。每个函数只做一件事，主程序只调用它们。

对比：改完之后你觉得拆之前和拆之后哪个更容易理解？

实验 5.2：函数调试题

下面这段代码会报错。先读，猜错在哪里，再运行验证。

```
def double(x):  
    return x * 2  
  
result = double(5)  
  
print(double) # 少写了什么?  
  
def add(a, b):  
    total = a + b  
  
result = add(3, 4)  
  
print(result) # 打印出来的是什么?
```

实验 5.3: 先写 docstring, 再写代码

选下面一个题目, 先写 docstring (描述函数做什么、参数是什么、返回什么), 再写代码。强制规则: 不先写完 docstring, 不允许写代码体。

1. `is_palindrome(s)`: 判断字符串是否是回文 (正读反读一样, 如 "radar")
2. `factorial(n)`: 计算 `n` 的阶乘

第 6 章实验: 容器玩法

实验 6.1: 列表变异观察 (必做)

先猜下面两段代码的输出, 再运行验证。思考为什么行为不一样。

```
a = [1, 2, 3]  
  
b = a  
  
b.append(4)  
  
print(a) # 你觉得是多少?  
  
x = 10  
  
y = x  
  
y = 20  
  
print(x) # 现在是多少?
```

思考: 为什么列表改了 `b` 会影响 `a`, 但数字改了 `y` 不影响 `x`? 这就是第二章"标签模型"的应用。

实验 6.2: 用字典做简单统计

给你一篇文章（自己写一段英文短文，或复制一段新闻标题），用字典统计每个字母出现的次数（忽略大小写，只统计 a-z）。

```
text = "Your text here..."

counts = {}

# 你的代码
```

扩展：改成统计单词出现的次数，去掉标点符号。

实验 6.3: 通讯录扩展

把第 6 章的通讯录程序加两个功能：

1. 删除联系人

2. 搜索联系人（输入关键词，如"张"找出所有姓名包含"张"的联系人）

提示：用 in 判断子串："张" in "张三" 返回 True。

第 7 章实验：跟文件对话

实验 7.1: 写一个日记程序

每次运行让你输入今天的感受，然后追加写到 diary.txt，每行加上日期。

```
from datetime import datetime

today = datetime.now().strftime("%Y-%m-%d %H:%M")
```

日记效果：

2026-06-08 20:30：今天学了文件操作，挺有意思

2026-06-09 09:15：下雨了，不想出门

实验 7.2: 通讯录 + 文件持久化

改造第 6 章的通讯录，让它：

启动时从文件 contacts.txt 读取联系人

退出时保存回文件

格式自定（每行"姓名,电话"最简单）

实验 7.3: 故意搞坏再修好

依次运行下面的代码，观察报错信息，然后加 `try-except` 修复，不能让它崩溃。

```
int("不是数字")
[1, 2, 3][100]
{"a": 1}["b"]
1 / 0
```

强制：修复每一个之后用 `print` 输出友好的错误提示。

综合挑战（学完第 7 章后做）

挑战 A: 命令行记账本

记录一笔收入（金额 + 备注）

记录一笔支出（金额 + 备注）

查看当前余额

查看最近 10 条记录

数据持久化到文件

规划：先不写代码，在一张纸上画出程序应该有哪些函数、数据怎么存、文件格式是什么。

挑战 B: 密码生成器

```
def generate_password(length=12, use_symbols=True):
```

生成随机密码，要求至少包含大写字母、小写字母、数字，如果 `use_symbols=True` 则包含 `!@#%$%^&*`。

提示：`import random; import string`

挑战 C: 代码考古

找一份你三个月前写的代码（或网上找一份简单的 Python 代码），尝试：

加 `docstring`

把超过 10 行的函数拆小

找到至少一个可能崩溃的地方，加 `try-except`

回答：改完之后的代码，好在哪里？

造梦者的下一行诗

浅尝到此。

这本书讲的内容并不深入，考虑到新手的适应性以及这本书的入门性，我们删去了后面一半的内容。在 Github 仓库，你应该也看到了那后半本的电子版。

如果你觉得有意思，可以继续往前走：

学习后半本的内容（面向对象编程是 Python 的核心范式）

或者，干脆就拿着现在的本事，去解决你身边一个真实的小麻烦——比如帮你妈整理通讯录，帮你朋友批量改文件名

编程这门手艺最迷人的地方不在于它有多难，而在于你写下一行代码，电脑就老实执行一行。每一次报错都是它在告诉你：这里还差一点。你改，它跑。就这么简单。

你把这本书从第一页看到了这里，已经比绝大多数只收藏从未翻开的人走得远多了。

以后你学习更多的内容，可以借助 AI（不要依赖 AI 写代码），可以多在网上论坛找教程（比如菜鸟教程）。

后记很短，

未来很长。

绘萤者



有些路，只能一个人走。
有些事，只能自己去面对。
愿你在迷茫时不失方向，
在黑暗中依然心怀光亮。

